



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

Università degli studi di Bari Aldo Moro

DIPARTIMENTO DI INFORMATICA
Corso di Laurea Triennale in Informatica

CASO DI STUDIO DEL CORSO IN INGEGNERIA DELLA CONOSCENZA

SISTEMA DI PREDIZIONE DELL'ENERGIA PRODOTTA DA IMPIANTI EOLICI

Repository GitHub: https://github.com/PalmDomenico/ICON_24_25

Autore: Palmisano Domenico 778538 d.palmisano34@studenti.uniba.it

Indice

1. Introduzione	4
2. Analisi dei dati	5
2.1. Grafici	5
2.1.1 Matrice di Correlazione	5
2.1.2. Grafico 3D a dispersione	7
2.2 Ragionamento probabilisti	10
2.2.1. Problema delle variabili continue	10
2.2.2. Costruzione della rete	11
2.2.3. Analisi dei Risultati Ottenuti	12
2.3 Conclusione	16
3. Apprendimento non supervisionato	18
3.1. Clustering	18
3.1.1. Hard Clustering	19
3.1.2 Soft clustering	21
3.2. Riduzione della dimensionalità	22
3.3. Conclusione	23
3. Ragionamento logico e Prolog	24
5. Apprendimento supervisionato	30
5.1. Workflow	32
6. Preparazione dei dati	34
5.1. Organizzazione dei dati	34
5.2. Normalizzazione dei dati	35
7. Ottimizzazione iperparametri	36
7.1. Algoritmi di Regressione Lineare	36
7.1.1. Parametro alpha	36
7.1.2. Parametro l1 ratio	37
7.2.1. Decision Tree Regressor	38
7.2.2. Random Forest Regressor	38
7.2. Algoritmi basati su Support Vector Machine	38
7.3. LSTM model	39

8. Confronto modelli	40
8.2. Analisi dei risultati	43
Riferimenti Bibliografici	43

1. Introduzione

Obiettivo

L'obiettivo del sistema è quello di **prevedere la quantità di energia prodotta da un impianto eolico** è comprendere la **relazione fra le diverse variabili metereologiche** acquisite e la **produzione di energia**.

La parte di predizione dell'energia prodotta viene supportata da diversi elementi:

- **Ragionamento probabilistico**
- **Una KB**
- **Apprendimento non supervisionato con clustering**

Il modello viene addestrato per prevedere la produzione di energia, ciò consente una stima accurata della potenza che sarà prodotta in determinati intervalli di tempo. Questo strumento risulta fondamentale per **ottimizzare la gestione dell'impianto**, migliorare l'affidabilità delle operazioni e supportare decisioni strategiche legate alla pianificazione energetica.

Dataset di riferimento

Il dataset principale utilizzato nel progetto è →

<https://www.kaggle.com/datasets/mubashirrahim/wind-power-generation-data-forecasting>

Il dataset è strutturato nel seguente modo:

- **time**: Indica il momento della misurazione.
- **temperature_2m**: Temperatura dell'aria misurata a 2 metri dal suolo (°C).
- **relativehumidity_2m**: Umidità relativa dell'aria a 2 metri dal suolo (%).
- **dewpoint_2m**: Punto di rugiada a 2 metri dal suolo (°C), ovvero la temperatura alla quale l'aria diventa satura di umidità.
- **windspeed_10m**: Velocità del vento a 10 metri dal suolo (m/s).
- **windspeed_100m**: Velocità del vento a 100 metri dal suolo (m/s) (altezza delle turbine eoliche)
- **winddirection_10m**: Direzione del vento a 10 metri dal suolo (°), misurata in gradi rispetto al nord.
- **winddirection_100m**: Direzione del vento a 100 metri dal suolo (°).
- **windgusts_10m**: Raffiche di vento a 10 metri dal suolo (m/s).
- **power**: Potenza prodotta dall'impianto eolico

Il dataset iniziale è strutturato in quattro file CSV distinti, ciascuno corrispondente a una specifica pala eolica. Tali file sono stati aggregati in un unico dataset per incrementare la variabilità dei dati durante la fase di addestramento (training) del modello. Durante l'operazione di fusione, è stato preservato l'identificativo della pala eolica di origine per garantire la tracciabilità del dato.

2. Analisi dei dati

La fase di analisi dei dati è stata condotta attraverso un approccio combinato di esplorazione visiva e modellazione probabilistica, esse hanno avuto lo scopo di identificare le relazioni tra le variabili meteorologiche e la produzione energetica, ma anche valutare la relazione fra le diverse variabili. Il tutto ha lo scopo di supportare la previsione di energia prodotta.

Nella prima sezione sono presenti grafici che vengono analizzati dal punto di vista visivo, successivamente viene fatta un'analisi probabilistica sul dataset, per infine trarre delle conclusioni sulle relazioni scoperte all'interno dei dati.

2.1. Grafici

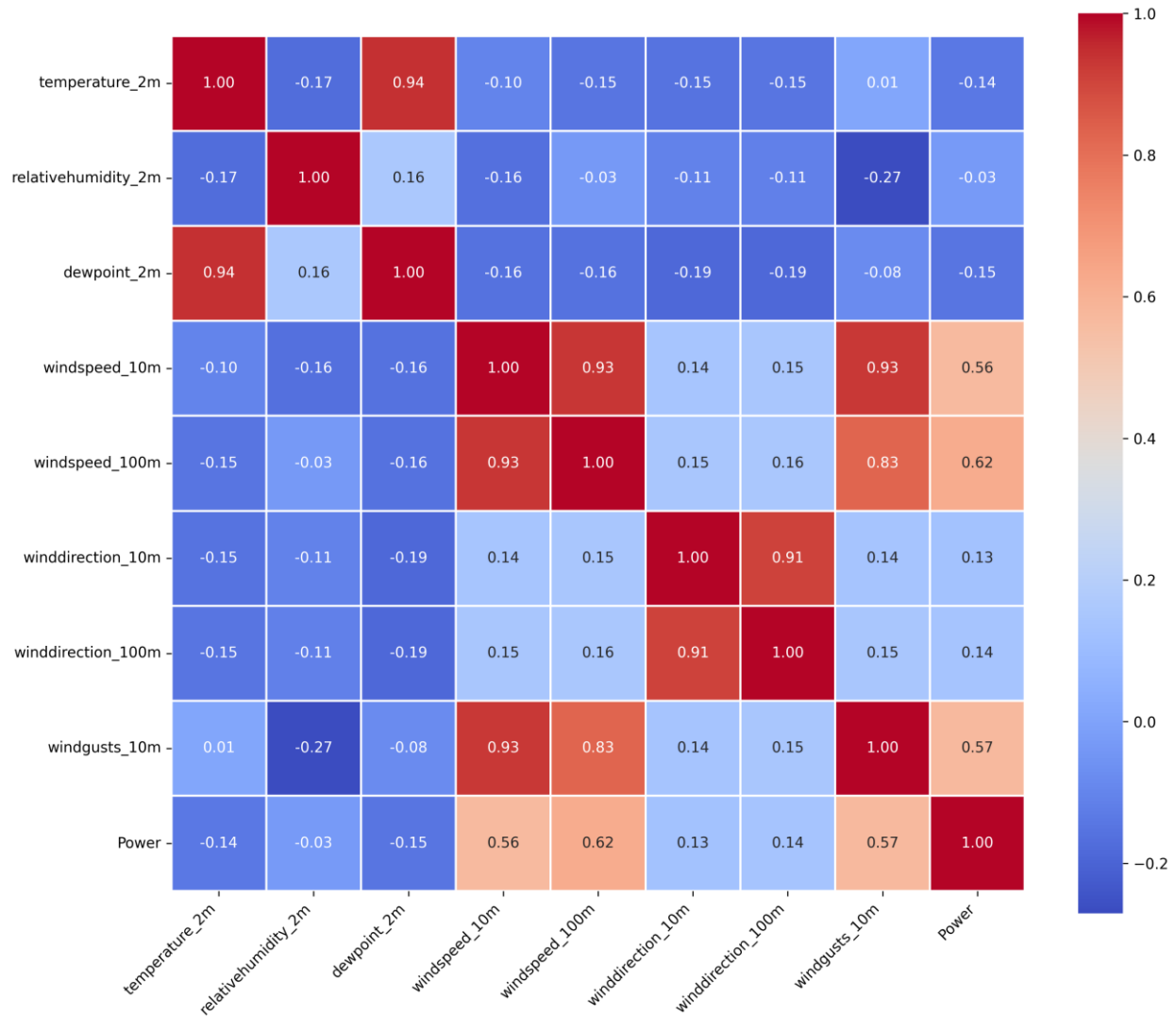
In primo luogo, è stata eseguita un'analisi esplorativa (EDA), il tutto è stato fatto attraverso la creazione di script python dedicati, i quali hanno generato 2 grafici principali:

- **Matrice di correlazione**
- **Grafico 3D a dispersione.**

2.1.1 Matrice di Correlazione

La heatmap rappresenta la **matrice di correlazione** tra le variabili del dataset, fornendo una visione immediata delle relazioni tra di esse. La correlazione misura il grado di dipendenza lineare tra due variabili, con valori compresi tra -1 e +1:

- **+1: Correlazione positiva perfetta** → quando una variabile aumenta, anche l'altra aumenta.
- **0 Nessuna correlazione lineare.**
- **-1 Correlazione negativa perfetta** → quando una variabile aumenta, l'altra diminuisce.



Correlazione tra la Potenza (Power) e le altre variabili

- **windspeed_100m (0.62)** → Forte correlazione positiva
 - Essa rappresenta la variabile con la correlazione più alta in assoluto con la potenza. Indica che la velocità del vento all'altezza del mozzo della turbina (100 metri) è colei più legata alla produzione energetica.
- **windgusts_10m (0.57)** → Correlazione positiva
 - Anche le raffiche di vento sono fortemente legata alla potenza
- **windspeed_10m (0.56)** → Correlazione positiva
 - La velocità del vento a bassa quota anch'essa risulta essere un buon indicatore.
- **winddirection_100m (0.14) e winddirection_10m (0.13)** → Correlazione positiva debole
 - La direzione del vento ha un impatto marginale sulla potenza totale prodotta.
- **temperature_2m (-0.14)** → Correlazione negativa debole

- Temperature più basse favoriscono una potenza leggermente maggiore.
- **dewpoint_2m (-0.15) →** Correlazione negativa debole
 - Simile alla temperatura, il punto di rugiada ha un impatto trascurabile ma negativo sulla produzione.
- **relativehumidity_2m (-0.03) →** Correlazione nulla
 - L'umidità relativa non ha praticamente alcuna relazione lineare con la potenza generata.

Analisi delle relazioni tra le variabili meteorologiche

- **Temperatura e Dewpoint (0.94) →** Correlazione altissima
 - temperature_2m e dewpoint_2m sono quasi identici.
- **Caratteristiche del Vento (0.93) →** Correlazione altissima
 - C'è una fortissima correlazione (0.93) tra windspeed_10m e windspeed_100m, così come tra windspeed_10m e windgusts_10m.
 - Questo significa che, se aumenta il vento a terra, aumenta quasi certamente anche in quota e aumentano le raffiche.
- **Direzione del Vento (0.91) →** Correlazione molto forte
 - La direzione del vento a 10m e a 100m è estremamente coerente.
- **Vento e Umidità (-0.27) →** Correlazione negativa moderata
 - Si nota una leggera correlazione negativa (-0.27) tra l'umidità relativa (relativehumidity_2m) e le raffiche (windgusts_10m).

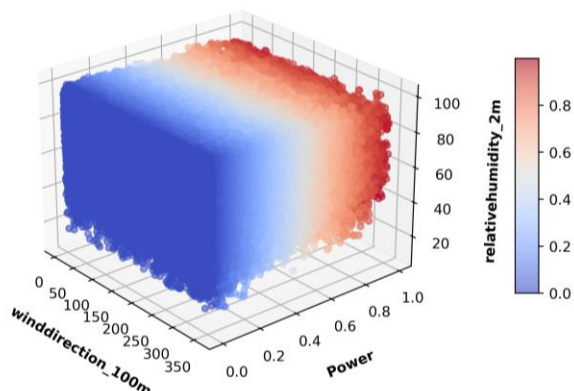
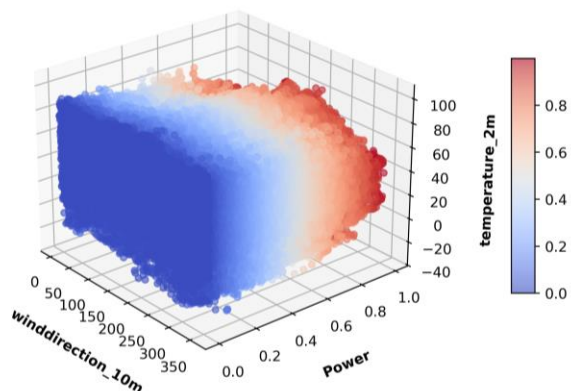
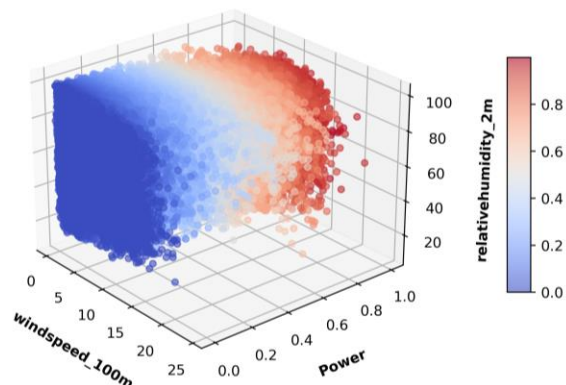
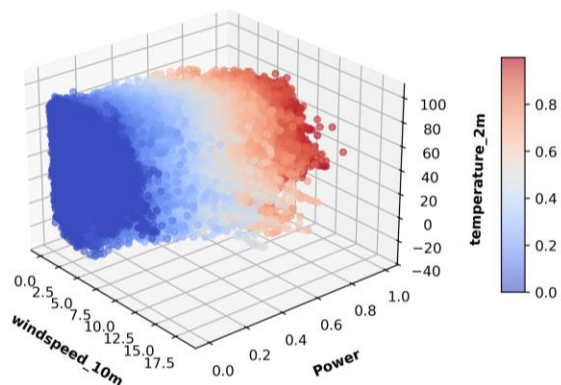
2.1.2. Grafico 3D a dispersione

Il grafico 3D a dispersione è progettato per visualizzare la **relazione tra tre variabili** contemporaneamente, facilitando la comprensione dell'**influenza di diverse condizioni atmosferiche e del vento sulla potenza prodotta** da un impianto eolico.

Per leggere il grafico, si devono osservare tre aspetti principali: l'asse X, l'asse Y e l'asse Z.

- **L'asse X** rappresenta una variabile legata al vento, come la velocità o la direzione del vento, e mostra come queste caratteristiche influenzano la potenza.
- **L'asse Y** rappresenta la potenza prodotta, che cambia in base ai valori delle altre due variabili.
- **L'asse Z**, mostra una variabile ambientale, come la temperatura o l'umidità, e il suo impatto sulla produzione di energia.

Ogni punto nel grafico rappresenta una combinazione di valori di queste variabili, e il colore dei punti indica l'**intensità della potenza prodotta**, con una scala che varia dal blu (bassa potenza) al rosso (alta potenza). Per interpretare il grafico, è importante osservare come i cambiamenti nella velocità del vento, nella direzione del vento o nelle condizioni atmosferiche influenzano la potenza, identificando eventuali tendenze o pattern che possono aiutare a ottimizzare la gestione dell'impianto eolico.



Analisi dei grafici:

- **Grafico 1.**
 - **Asse X (in basso a sinistra):** *windspeed_10m*.
 - **Asse Y (in basso a destra):** *Power*
 - **Asse Z (verticale):** *temperature_2m*
 - **Colore:** Rappresenta la *temperature_2m* (Blu = freddo, Rosso = caldo).
 - **Analisi:** Si vede chiaramente che all'aumentare della velocità del vento, la potenza sale.
- **Grafico 2.**
 - **Asse X (in basso a sinistra):** *windspeed_100m*
 - **Asse Y (in basso a destra):** *Power*.
 - **Asse Z (verticale):** *relativehumidity_2m*
 - **Colore:** Rappresenta *relativehumidity_2m* (Blu = secco, Rosso = umido).
 - **Analisi:** Questo mostra come la velocità a 100m influisce direttamente la potenza prodotta. Inoltre, si vede come sopra i 12-13 m/s la curva si appiattisce (la turbina raggiunge la potenza massima e non sale oltre).

- **Grafico 3.**
 - **Asse X (in basso a sinistra):** *winddirection_10m*.
 - **Asse Y (in basso a destra):** *Power*.
 - **Asse Z (verticale):** *temperature_2m*.
 - **Colore:** Rappresenta la *temperature_2m* (Blu = freddo, Rosso = caldo).
 - **Analisi:** Questo mostra come indipendentemente dalla direzione del vento la turbina gira e produce energia indipendentemente da dove arriva il vento, questo è dovuto magari ad un adattamento della direzione della turbina in base alla direzione del vento.
- **Grafico 4.**
 - **Asse X (in basso a sinistra):** *winddirection_100m*.
 - **Asse Y (in basso a destra):** *Power*.
 - **Asse Z (verticale):** *relativehumidity_2m*.
 - **Colore:** Rappresenta *relativehumidity_2m* (Blu = secco, Rosso = umido).
 - **Analisi:** Simile al grafico precedente, esso conferma l'indipendenza fra l'energia prodotta e la direzione del vento in questo caso a 100m.

2.2 Ragionamento probabilisti

Il ragionamento probabilistico è un paradigma che applica la teoria della probabilità per gestire l'incertezza. Si basa su concetti fondamentali come la dipendenza e l'indipendenza tra variabili e sfrutta la **Regola di Bayes** come motore inferenziale.

Il processo logico segue due passaggi principali:

1. **Assegnazione:** Si attribuiscono probabilità iniziali a ipotesi o eventi.
2. **Aggiornamento:** Si calcolano le *probabilità a posteriori* per trarre conclusioni basate sulle evidenze disponibili.

Lo strumento principale per applicare questo tipo di ragionamento è la **Rete Bayesiana**. Le reti bayesiane sono modelli grafici strutturati come **Grafi Orientati Aciclici (DAG)**, composti da due elementi:

- **Nodi:** Rappresentano le variabili del sistema.
- **Archi:** Indicano le dipendenze probabilistiche dirette tra le variabili.

Nel nostro caso;

- **Nodi:** Rappresentano le variabili del dataset come *windspeed_100m* o *Power*.
- **Archi:** Un arco *Vento* → *Potenza* indica che la variabile vento influenza direttamente la produzione.

Inoltre, abbiamo le CPD (Conditional Probability Distributions), ovvero tabelle che quantificano la probabilità di ogni stato di un nodo, dati gli stati dei suoi nodi genitori.

2.2.1. Problema delle variabili continue

Le Reti Bayesiane presentano limiti strutturali nella gestione di feature continue. La costruzione delle Tabelle di Probabilità Condizionata (CPD) richiede l'osservazione di ogni possibile valore; di conseguenza, l'inferenza su nuovi dati contenenti valori non presenti nel *training set* ha generato diversi errori. La soluzione è quindi stata quella di applicare la discretizzazione delle variabili continue tramite *KBinsDiscretizer*. Questo approccio ha prodotto a diversi benefici fondamentali, ovvero la robustezza dal momento che ha eliminato gli errori di predizione su istanze non osservate e l'efficienza dal momento che ha risolto le criticità legate al consumo di memoria.

2.2.2. Costruzione della rete

Le Reti Bayesiane discrete richiedono variabili categoriche o intervalli finiti. Poiché i dati meteorologici e di potenza sono continui, abbiamo applicato una **discretizzazione** utilizzando KBinsDiscretizer con strategia uniforme:

```
def preprocess_and_discretize(df, n_bins=5):  
    """Discretizes continuous variables into ordinal bins."""  
    discretizer = KBinsDiscretizer(n_bins=n_bins, encode='ordinal', strategy='uniform')  
    data_discrete = discretizer.fit_transform(df)  
    df_discrete = pd.DataFrame(data_discrete, columns=df.columns).astype(int).astype(str)  
    return df_discrete
```

Questa trasformazione converte i valori continui in 5 classi ordinate da 0 a 4, permettendo alla rete di calcolare le tabelle di probabilità condizionata (CPD) su stati finiti (es. *Vento Basso, Medio, Alto*).

La fase successiva è stata quella dell'apprendimento della Struttura

Per definire la topologia della rete senza introdurre bias manuali, abbiamo utilizzato l'apprendimento automatico della struttura dai dati.

```
def train_bayesian_model(df_discrete, max_iter=1000):  
    hc = HillClimbSearch(df_discrete)  
    structure = hc.estimate(scoring_method='k2', max_iter=max_iter)  
    model = DiscreteBayesianNetwork(structure.edges())  
    model.fit(  
        df_discrete,  
        estimator=BayesianEstimator,  
        prior_type='BDeu',  
        equivalent_sample_size=10  
    )  
    return model
```

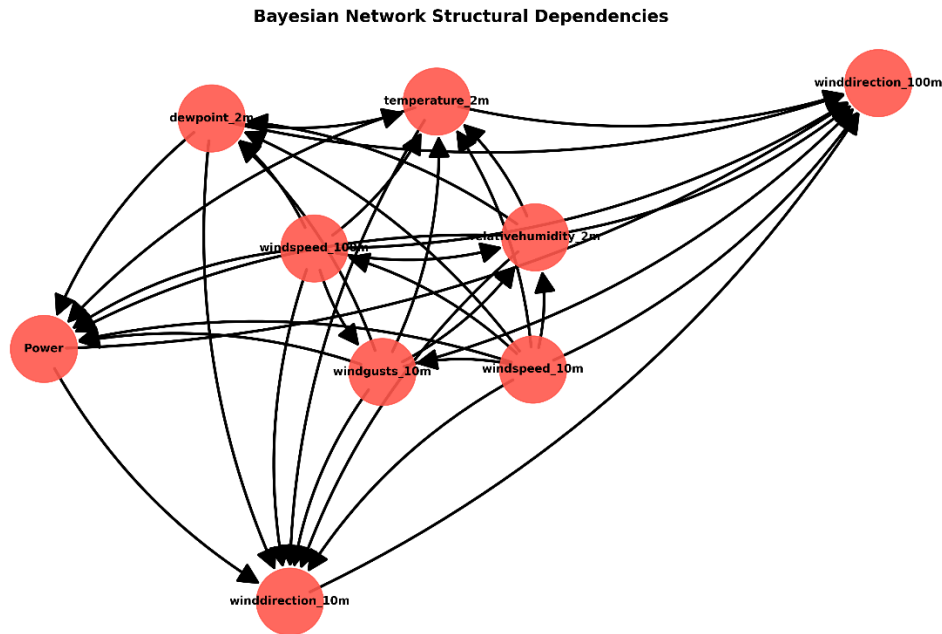
Come si può vedere nel metodo `train_bayesian_model`, utilizziamo l'algoritmo Hill Climb Search per l'apprendimento della struttura. L'algoritmo esplora lo spazio delle possibili strutture per massimizzare il punteggio **K2**, una metrica bayesiana che valuta quanto bene il modello spiega i dati osservati, penalizzando strutture troppo complesse per prevenire l'overfitting.

A differenza di approcci più semplici (come la Maximum Likelihood), abbiamo scelto di utilizzare un **Bayesian Estimator** per l'apprendimento delle probabilità (CPD). L'uso del BayesianEstimator con prior BDeu (Bayesian Dirichlet equivalent uniform) introduce un meccanismo di *smoothing* (pseudo-conteggi). Questo è cruciale per gestire combinazioni di eventi rari che potrebbero non comparire nel dataset di training, evitando che la rete assegni probabilità zero (problema della scarsità dei dati) il che rende il modello più robusto.

2.2.3. Analisi dei Risultati Ottenuti

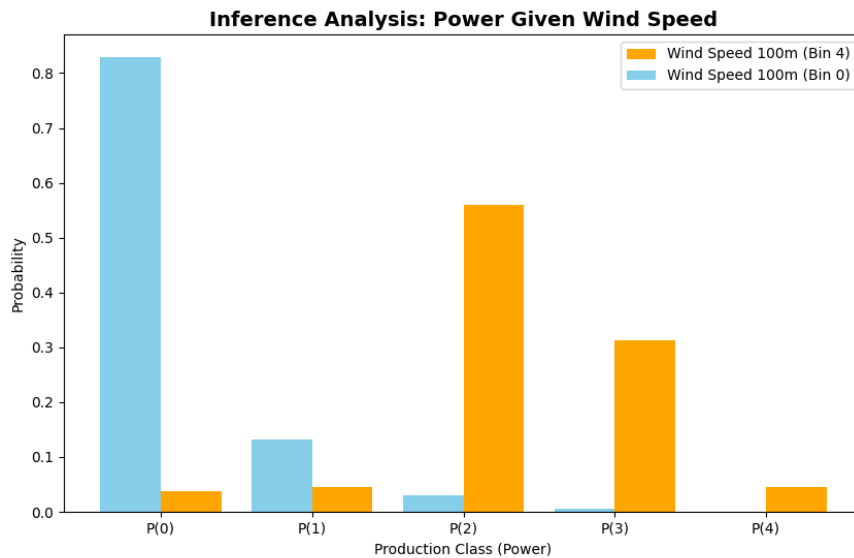
2.2.3.1. Rete bayesiana

Come primo elemento da analizzare abbiamo la struttura della rete.



Il grafo evidenzia le dipendenze già precedentemente apprese sia con i grafici a dispersione che con le heatmap. È possibile osservare come il nodo Power sia direttamente influenzato da molte variabili chiave come windspeed_100m, la quale abbiamo detto che è la principale influenza della produzione di energia. La presenza di archi tra le variabili meteorologiche (es. tra windspeed e windgusts) conferma la capacità della rete di catturare anche ulteriori dipendenze fra i dati.

2.2.3.2. Impatto della Velocità del Vento



Il grafico confronta la distribuzione di probabilità della *Power* in due scenari opposti: bassa velocità del vento (Bin 0, barre azzurre) e alta velocità del vento (Bin 4, barre arancioni).

- Quando il vento è basso (Bin 0), la probabilità che la potenza sia nel Bin 0 (produzione minima) è quasi del 100%.
- Quando il vento è alto (Bin 4), la distribuzione si sposta drasticamente verso destra, con la massima probabilità concentrata nel Bin 4 (produzione massima).

2.2.3.3. Tabelle CPD

I diversi accoppiamenti per la creazione delle tabelle CPD sono stati scelti in base ad analisi fatte anche sulla rete bayesiana precedentemente ottenuta. La velocità del vento a 100m è stata scelta come termine di paragone fisso in quanto abbiamo notato essere quella che più influenza l'energia prodotta. L'incrocio con la temperatura serve a verificare che il modello rispetti la legge fisica per cui l'aria fredda aumenta la potenza, mentre l'analisi della direzione è necessaria per escludere ostacoli geografici e confermare l'efficienza dell'orientamento della turbina. Infine, il confronto con le raffiche è utile per capire se il vento con turbolenze aiuta la produzione o meno.

Potenza vs Velocità Vento (100m) e Temperatura

State windspeed_100m	State temperature_2m	P(Power=0)	P(Power=1)	P(Power=2)	P(Power=3)	P(Power=4)
windspeed_100m(0)	temperature_2m(0)	0.8632	0.1248	0.0117	0.0002	0.0002
windspeed_100m(0)	temperature_2m(2)	0.8042	0.1509	0.0366	0.0069	0.0015
windspeed_100m(0)	temperature_2m(4)	0.8763	0.0970	0.0214	0.0051	0.0002
windspeed_100m(2)	temperature_2m(0)	0.0545	0.3195	0.3477	0.2152	0.0632
windspeed_100m(2)	temperature_2m(2)	0.1043	0.1189	0.1996	0.3279	0.2494
windspeed_100m(2)	temperature_2m(4)	0.1810	0.1647	0.3074	0.2552	0.0918
windspeed_100m(4)	temperature_2m(0)	0.0108	0.0108	0.9568	0.0108	0.0108
windspeed_100m(4)	temperature_2m(2)	0.0648	0.0648	0.3837	0.4219	0.0648
windspeed_100m(4)	temperature_2m(4)	0.2000	0.2000	0.2000	0.2000	0.2000

Scopo della tabella: *Come influisce la temperatura sulla produzione a parità di vento?*

La tabella incrocia tre stati chiave (0=Basso, 2=Medio, 4=Alto) per la velocità del vento e la temperatura.

- **L'Influenza del Vento**

- **Quando windspeed_100m è 0 (Basso)**, osserviamo che la probabilità di Power=0 è prossima al **100% (1.0)**, indipendentemente dalla temperatura (sia essa 0, 2 o 4). Quindi ci fa comprendere che senza vento, la turbina non gira, la temperatura è irrilevante a prescindere.

- Quando **windspeed_100m** è 4 (Alto), la probabilità si sposta massicciamente su Power=4.
- **L'Influenza della Temperatura**
 - Confrontando Temperature=0 (Freddo) vs Temperature=4 (Caldo), notiamo una **leggera variazione** nelle probabilità. L'aria fredda è più densa. Pertanto, a parità di velocità del vento (es. stato '2'), una temperatura bassa (0) mostra una probabilità leggermente superiore di produrre potenza elevata rispetto a una temperatura calda.

Potenza vs Velocità Vento (100m) e Direzione

State windspeed_100m	State winddirection_100m	P(Power=0)	P(Power=1)	P(Power=2)	P(Power=3)	P(Power=4)
windspeed_100m(0)	winddirection_100m(0)	0.8568	0.1163	0.0226	0.0043	0.0000
windspeed_100m(0)	winddirection_100m(2)	0.8132	0.1425	0.0358	0.0073	0.0013
windspeed_100m(0)	winddirection_100m(4)	0.8180	0.1423	0.0317	0.0070	0.0009
windspeed_100m(2)	winddirection_100m(0)	0.1212	0.1413	0.1820	0.3577	0.1977
windspeed_100m(2)	winddirection_100m(2)	0.1461	0.1390	0.2423	0.2832	0.1895
windspeed_100m(2)	winddirection_100m(4)	0.1461	0.1482	0.2146	0.3106	0.1804
windspeed_100m(4)	winddirection_100m(0)	0.0095	0.0095	0.6048	0.3667	0.0095
windspeed_100m(4)	winddirection_100m(2)	0.0049	0.0659	0.6756	0.1878	0.0659
windspeed_100m(4)	winddirection_100m(4)	0.0230	0.0562	0.5872	0.3106	0.0230

Scopo della tabella: *Esistono direzioni del vento privilegiate per la produzione?*

- Mantenendo fissa la velocità del vento (es. a livello 4), osserviamo che al variare della direzione (0, 2, 4), la distribuzione di probabilità della Potenza rimane **pressoché identica**. Questo conferma le precedenti analisi che ci conferma come la turbina si orienta per seguire il vento.

3. Analisi: Potenza vs Velocità Vento (100m) e Raffiche (10m)

State windspeed_100m	State windgusts_10m	P(Power=0)	P(Power=1)	P(Power=2)	P(Power=3)	P(Power=4)
windspeed_100m(0)	windgusts_10m(0)	0.8343	0.1286	0.0301	0.0060	0.0010
windspeed_100m(0)	windgusts_10m(2)	0.4478	0.3182	0.1164	0.1020	0.0156
windspeed_100m(0)	windgusts_10m(4)	0.2000	0.2000	0.2000	0.2000	0.2000
windspeed_100m(2)	windgusts_10m(0)	0.3277	0.3277	0.2213	0.1149	0.0085
windspeed_100m(2)	windgusts_10m(2)	0.1192	0.0983	0.1827	0.3392	0.2606
windspeed_100m(2)	windgusts_10m(4)	0.0235	0.0235	0.3176	0.0235	0.6118
windspeed_100m(4)	windgusts_10m(0)	0.2000	0.2000	0.2000	0.2000	0.2000
windspeed_100m(4)	windgusts_10m(2)	0.0571	0.0571	0.7714	0.0571	0.0571
windspeed_100m(4)	windgusts_10m(4)	0.0779	0.0779	0.2687	0.4977	0.0779

Scopo della tabella: *Le raffiche aggiungono potenza o sono solo turbolenza?*

Qui analizziamo la relazione tra il vento medio in quota (windspeed_100m) e le raffiche a terra (windgusts_10m).

- Quando sia la velocità media che le raffiche sono alte (4 e 4), la probabilità di Power=4. Questo ci dice che è **improbabile avere vento medio alto e raffiche basse**.

2.3 Conclusione

Mettendo a confronto ciò che abbiamo visto nei grafici 3D con i calcoli della Rete Bayesiana, emerge una perfetta coerenza tra i due risultati. L'analisi visiva attraverso i grafici ci mostra che solo la velocità del vento impattava fortemente la produzione di energia, mentre l'analisi probabilistica ha confermato esattamente questo, assegnando percentuali di produzione alte solo quando il vento è forte. Questi risultati risultano particolarmente interessanti dal momento

che oltre alla produzione di energia si potrebbero creare sistemi di previsione della velocità del vento per poi determinare attraverso formule matematiche o reti bayesiane il possibile **range di produzione atteso**.

3. Apprendimento non supervisionato

Come abbiamo appreso nelle sezioni precedenti, determinate variabili influenzano **direttamente il livello di produzione dell'impianto**. Per questo ci viene in supporto l'apprendimento non supervisionato. Attraverso esso riusciremo a vedere come un **modello (KMeans) riesca a riconoscere condizioni meteo simili fra loro**, raggrupparle e associare ad essi un determinato cluster.

L'apprendimento non supervisionato è una branca dell'apprendimento automatico in cui l'agente viene addestrato su un insieme di dati senza etichette oppure senza un target. Esistono due principali tipi di apprendimento non supervisionato con diversi obiettivi:

- **Clustering** → L'obiettivo è raggruppare gli elementi del dataset in base a delle somiglianze.
- **Riduzione della dimensionalità** → L'obiettivo è ridurre il numero di feature utilizzate mantenendo però le informazioni più significative. Un esempio è la PCA (Principal Component Analysis).

3.1. Clustering

Attraverso il clustering andremo a raggruppare in n cluster condizioni meteo simili fra loro. Esistono due tipologie di apprendimento di clustering:

- **hard clustering** → Associa ogni esempio a un cluster specifico.
- **soft clustering** → Associa a ogni esempio la probabilità di appartenenza a ogni cluster.

Le due tipologie di clustering vengono messe a confronto. Esse saranno confrontate andando a confrontare i grafici che analizzano la distribuzione della potenza per cluster, ma anche tramite una riduzione dimensionale tramite PCA per valutare visivamente la separazione tra i cluster.

Scopo

Lo scopo principale di usare tecniche di clustering è quello di **trovare condizioni meteo simili fra loro** e successivamente scovare relazioni fra alcune condizioni meteo e la produzione di energia.

Andiamo a vedere come soft e hard clustering si comportano e andremo a trarre delle conclusioni su quella che è la tecnica migliore da usare in questo caso.

Sia per l'hard clustering che per il soft clustering sono stati seguiti i seguenti step:

- Normalizzazione dei dati.

```
scaler = StandardScaler()
features = ['temperature_2m', 'relativehumidity_2m', 'dewpoint_2m',
            'windspeed_10m', 'windspeed_100m',
            'winddirection_10m', 'winddirection_100m',
            'windgusts_10m', 'Power']
X = dataset[features]
X_scaled = scaler.fit_transform(X)
```

- Per fare ciò viene usata la libreria sklearn.

- Ricerca del numero di cluster ottimale da utilizzare.
- Allenamento del modello.

3.1.1. Hard Clustering

L'hard clustering è una tecnica di clustering in cui ogni punto del dataset viene assegnato in **modo esclusivo a un solo cluster**. Ad esempio l'algoritmo KMeans assegna a ciascun punto il centroide più vicino. In questo caso appunto viene usato l'algoritmo KMeans.

Uno dei problemi principali del KMeans è **trovare il numero ideale di cluster** da fornire in input all'algoritmo. Per risolvere questo problema è stata usata una strategia nota come "curva del gomito". La curva del gomito mostra la **variazione dell'inertia al variare del numero di cluster**, dove l'inertia rappresenta la somma delle distanze quadrate tra ogni punto dati e il centro del cluster assegnato.

L'algoritmo prevede di eseguire KMeans per diversi valori del numero di cluster sulla quale viene calcolata l'**inertia**. L'obiettivo è identificare il cosiddetto **"gomito" (elbow point)**, ovvero il punto a partire dal quale l'incremento del numero di cluster comporta una riduzione dell'inertia sempre meno significativa. Questo punto rappresenta una buona scelta per il numero ottimale di cluster.

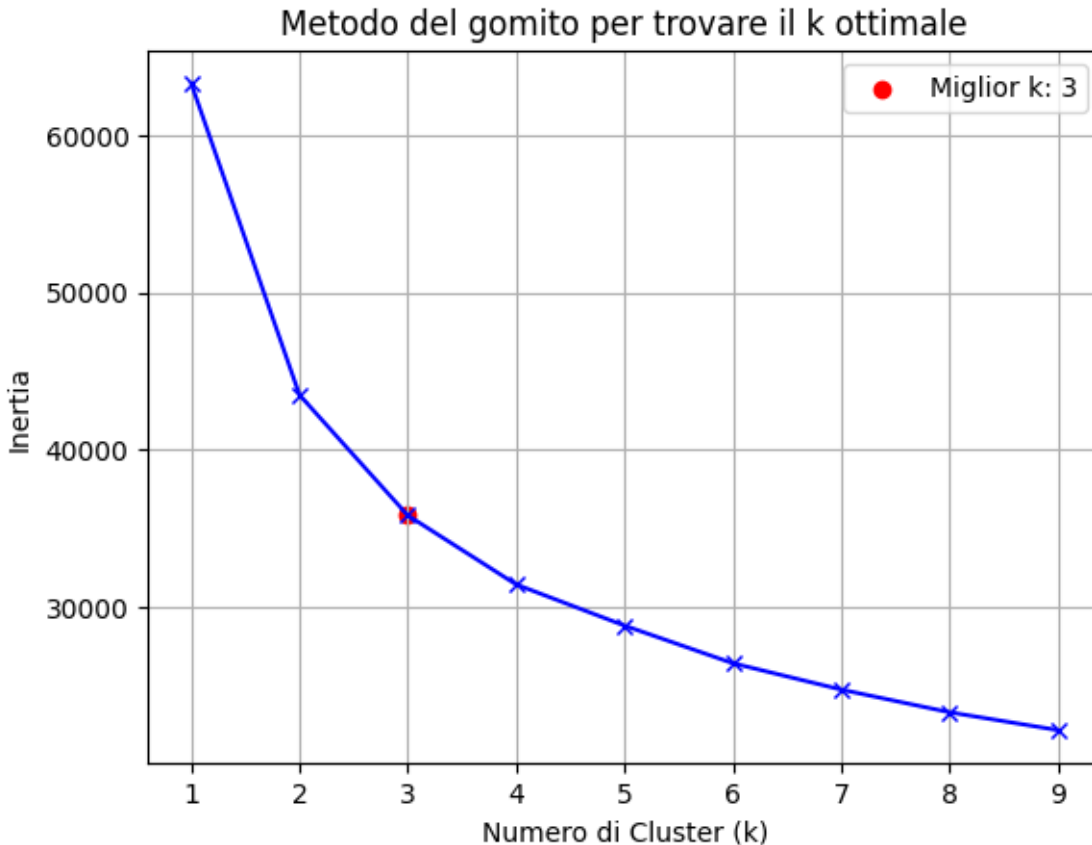
```
iterate = []
for i in range(1, maxK):
    kmeans = KMeans(n_clusters=i, n_init=10, init='random')
    kmeans.fit(dataSet)
    iterate.append(kmeans.inertia_)

kl = KneLocator(range(1, maxK), iterate, curve="convex", direction="decreasing")
```

PARAMETRI:

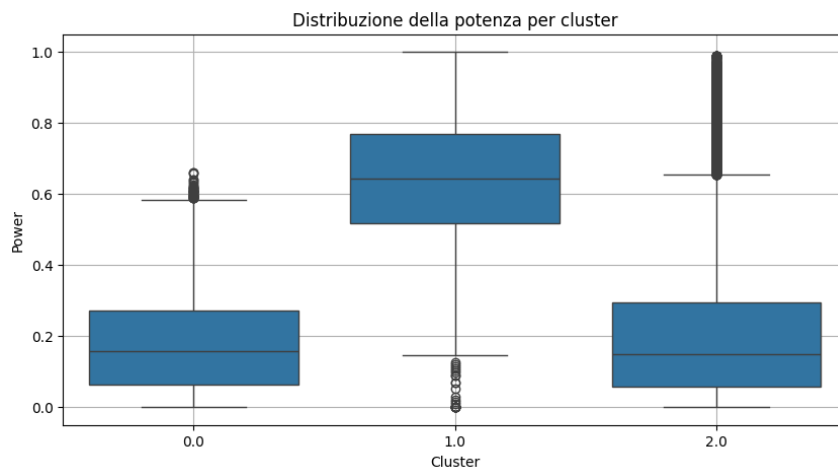
- L'algoritmo viene eseguito con 10 valori di cluster, che variano da 1 a 10.
- Il parametro *n_init* indica quante volte verrà eseguito l'algoritmo sul cluster i, il quale una volta concluso restituirà il risultato migliore degli *n_init* allenamenti.
- Il parametro *init='random'* significa che i centroidi vengono inizializzati in maniera del tutto casuale.

L'algoritmo è stato testato su diversi dataset simili nella forma ma contenente dati diversi. Una volta eseguito l'algoritmo otteniamo un grafico che ci indica il numero ideale di cluster da utilizzare per quello specifico dataset.



Conclusa la parte precedente alleniamo l'algoritmo KMeans sul relativo k trovato per i differenti dataset. Per analizzare la **correlazione che c'è fra determinate condizioni meteo e potenza prodotta**, analizziamo i seguenti grafici che mostrano come i diversi cluster hanno range di potenza differenti.

- **Cluster 1** → ha la potenza mediamente più alta, con valori centrali attorno a 0.6–0.8.
- **Cluster 0 e Cluster 2** → mostrano distribuzioni simili, con mediane vicine a 0.1 e una maggiore dispersione.



Possiamo notare come anche se ci sono cluster con range di potenza simile fra loro, c'è sempre un cluster che si differenzia molto dagli altri. In conclusione, possiamo dire che ci sono

determinate condizioni meteo che sono favorevoli alla produzione di energia e altre meno favorevoli, che non sempre è dovuta ad un'assenza totale di vento ma potrebbe derivare anche da condizioni instabili o fattori simili.

3.1.2 Soft clustering

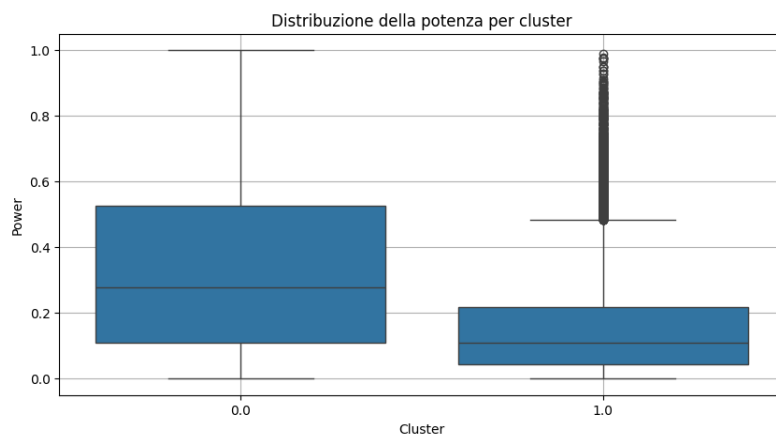
Il soft clustering è una tecnica in cui ogni punto può appartenere a più cluster con un certo grado di probabilità. Invece di un'assegnazione netta, come nell'hard clustering, si lavora con appartenenze parziali. Un esempio tipico è il Gaussian Mixture Model (GMM), che calcola la probabilità che un punto appartenga a ciascun cluster in base alla distribuzione statistica.

Come per l'hard clustering, anche qui è necessario ricercare il miglior numero di cluster da utilizzare.

```
def research_n_clusters_gmm(X_scaled, max_k=10):
    best_k = 2
    best_score = -1
    for k in range(2, max_k + 1):
        gmm = GaussianMixture(n_components=k, n_init=10, random_state=0)
        labels = gmm.fit_predict(X_scaled)
        score = silhouette_score(X_scaled, labels)
        if score > best_score:
            best_k = k
            best_score = score
    return best_k
```

Questa funzione prova valori di k da 2 a max_k ed , calcola il silhouette score e restituisce il k con il punteggio migliore. La **silhouette score** è una metrica che valuta la qualità di un clustering.

Conclusa la parte precedente alleniamo l'algoritmo Gaussian Mixture Model sul rispettivo numero di cluster trovati.

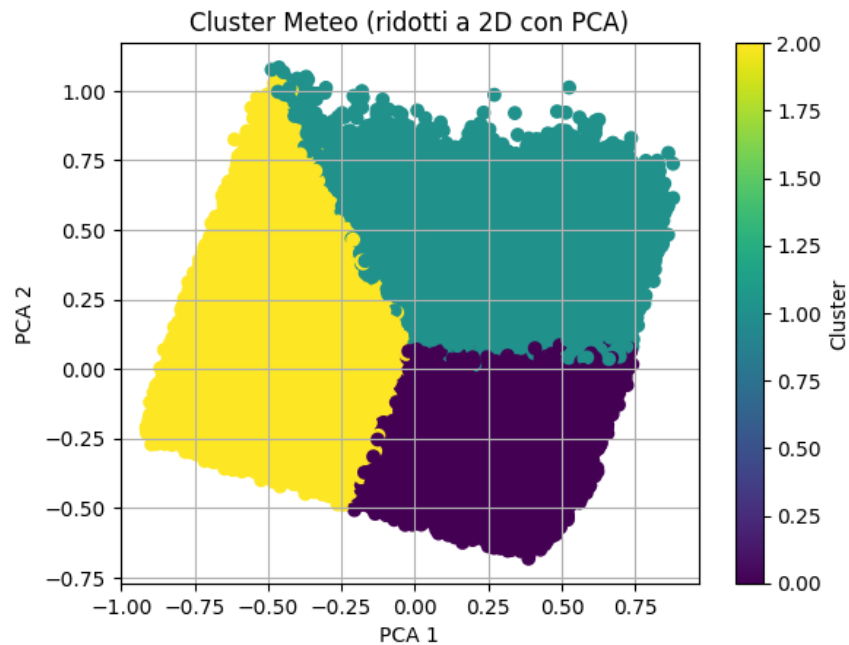


In questo caso vediamo come i 2 cluster sono parzialmente sovrapposti e non c'è un cluster che si differenzia molto dagli altri, come succede nel hard clustering.

3.2. Riduzione della dimensionalità

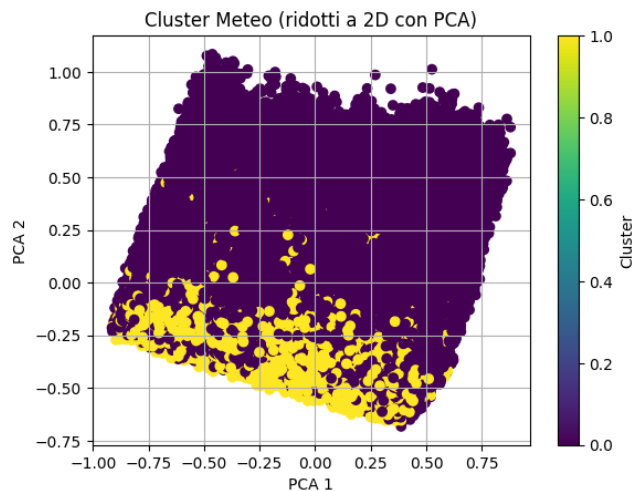
La **Principal Component Analysis (PCA)** è una tecnica di riduzione dimensionale che consente di trasformare un dataset ad alta dimensionalità in un nuovo spazio con meno variabili (componenti principali), mantenendo la maggior parte della varianza informativa. Nel nostro caso, i dati meteorologici sono stati proiettati su due componenti principali per facilitarne la visualizzazione.

Kmeans



Il grafico mostra una chiara separazione tra i cluster nello spazio 2D delle componenti principali, indicando che i gruppi individuati rappresentano **condizioni meteorologiche ben distinte**. La distribuzione compatta e ordinata dei punti conferma l'efficacia del clustering.

GMM



Il grafico mostra una **separazione meno netta** tra i cluster nello spazio 2D delle componenti principali, con una significativa sovrapposizione tra i gruppi. Questo indica che i gruppi individuati rappresentano **condizioni meteorologiche che si sovrappongono**.

3.3. Conclusione

Lo scopo principale per la quale è stato usato il clustering è quello di aggiungere informazioni utili al dataset. In conclusione, viene scelto l'hard clustering dal momento che esso ha mostrato una migliore adesione ai dati ed un'individuazione più netta di condizioni meteo simili fra essi, ovvero una distinzione abbastanza definita fra i vari cluster. Il dataset originale viene quindi arricchito di informazioni, i quali saranno utili a raggiungere prestazioni migliori ed individuare più facilmente pattern all'interno dei dati.

3. Ragionamento logico e Prolog

Il ragionamento logico si differenzia da quello probabilistico per il fatto che opera sempre in un contesto deterministico. Il ragionamento logico sfrutta la **logica** matematica. Nel ragionamento logico si costruisce una **Knowledge Base (KB)**, ovvero una base di conoscenza composta da **assiomi**, che rappresentano affermazioni considerate sempre vere.

Gli assiomi si dividono in:

- **Fatti**: rappresentano la conoscenza primitiva.
- **Regole**: permettono di derivare conoscenza derivata, che si ottiene a partire da osservazioni o inferenze.

Il modulo di ragionamento logico ha l'obiettivo di arricchire il dataset meteorologico attraverso l'inferenza di nuove feature basate su conoscenza di dominio (Knowledge Base). Questa fase sfrutta il paradigma della programmazione logica per derivare indicatori complessi che non sono esplicitamente presenti nei dati grezzi, ma che influenzano significativamente la produzione di energia eolica.

L'integrazione tra l'ambiente Python (utilizzato per il Machine Learning) e il motore logico Prolog avviene tramite la libreria pyswip. Il processo si articola in tre fasi principali gestite dallo script KB/prolog.py:

1. **Generazione dei Fatti**: I dati grezzi del dataset e i risultati del clustering vengono tradotti dinamicamente in fatti Prolog.
2. **Interrogazione della Knowledge Base**: Viene eseguita una query per calcolare le nuove feature per ogni istanza temporale.
3. **Arricchimento del Dataset**: I risultati dell'inferenza vengono mappati in nuove colonne nel dataset originale.

La **Knowledge Base** è composta da due file principali:

- *facts.pl (generato dinamicamente)*
- *rules.pl (regole statiche).*

Fatti

Il sistema scrive due tipi di fatti per ogni osservazione $r\{\text{Index}\}$:

- **Dati Meteorologici**: `meteo_data(ID, Temp, RH, Dew, W10, W100, D10, D100, Gust, Power)`.
 - **ID**: Identificativo univoco della riga (es. `r0`, `r1...`).
 - **Temp, RH, Dew**: Temperatura (2m), Umidità Relativa, Punto di Rugiada.
 - **W10, W100**: Velocità del vento a 10m e 100m.
 - **D10, D100**: Direzione del vento a 10m e 100m.
 - **Gust**: Raffiche di vento.

- **Power:** Potenza generata (variabile target).
- **Clustering:** cluster(ID, ClusterLabel).
 - Associa ogni record al cluster di appartenenza identificato nella fase di apprendimento non supervisionato.

Lo script utilizzato per scrivere i fatti dinamicamente è il seguente:

```
def write_facts(df_raw, df_cluster, filename):
    if os.path.exists(filename):
        os.remove(filename)

    facts = []
    for row in df_raw.iteruples(index=True):
        try:
            idx = row.Index
            t = getattr(row, 'temperature_2m', 0)
            rh = getattr(row, 'relativehumidity_2m', 0)
            dew = getattr(row, 'dewpoint_2m', 0)
            w10 = getattr(row, 'windspeed_10m', 0)
            w100 = getattr(row, 'windspeed_100m', 0)
            d10 = getattr(row, 'winddirection_10m', 0)
            d100 = getattr(row, 'winddirection_100m', 0)
            g10 = getattr(row, 'windgusts_10m', 0)
            p = getattr(row, 'Power', 0)

            # meteo_data(id, temp, rh, dew, w10, w100, d10, d100, gust, power).
            facts.append(
                f"meteo_data(r{idx}, {t:.2f}, {rh:.2f}, {dew:.2f}, {w10:.2f}, {w100:.2f}, {d10:.2f}, {d100:.2f}, {g10:.2f}, {p:.2f})."
            )
        except:
            continue

    with open(filename, 'w', encoding='utf-8') as f:
        f.write('\n'.join(facts))
        f.write('\n')

    # Write data cluster
    if 'Cluster' in df_cluster.columns:
        cluster_facts = []
        for row in df_cluster.iteruples(index=True):
            cluster_facts.append(f"cluster(r{row.Index}, {int(row.Cluster)}).")
        f.write('\n'.join(cluster_facts))
```

Regole

Il file *rules.pl* definisce i predicati per estrarre le nuove feature. Le regole implementate sono le seguenti:

- **calc_wind_potential**
 - Stima la percentuale di potenza producibile basandosi sulla velocità del vento a 100m(V), simulando la curva di potenza di una turbina standard:
 - **Cut-in speed:** Se $V < 3.0\text{m/s}$, il potenziale è **0.0**.
 - **Cut-out speed:** Se $V > 25.0\text{m/s}$, il potenziale è **0.0** → la turbina si ferma per sicurezza.
 - **Rated speed:** Se $V \geq 12.0\text{m/s}$, il potenziale è **1.0** (massima potenza).

```

calc_wind_potential(V, P) :-
    V < 3.0, P is 0.0, !.
calc_wind_potential(V, P) :-
    V > 25.0, P is 0.0, !.
calc_wind_potential(V, P) :-
    V >= 12.0, P is 1.0, !.
calc_wind_potential(V, P) :-
    Base is (V - 3.0) / 9.0,
    P is Base * Base * Base.

```

- **get_density_score**

- Le turbine producono più energia con aria densa (fredda).

```

get_density_score(X, Raw) :-
    get_temp(X, T),
    Diff is 15.0 - T,
    Correction is Diff * 0.003,
    Raw is 1.0 + Correction.

```

- **get_turbulence_idx**

- Misura la stabilità del vento confrontando le raffiche con la velocità media a 100m.

```

get_turbulence_idx(X, Index) :-
    get_gusts(X, G), get_wind100(X, V),
    V > 1.0,
    Index is G / V, !.
get_turbulence_idx(_, 0.0).

```

- **get_shear_index**

- Misura la variazione della direzione del vento tra la base di 10m e l'altezza di 100m.

```

get_shear_index(X, DiffDegrees) :-
    get_wind_dir10(X, D10), get_wind_dir100(X, D100),
    Delta is abs(D100 - D10),
    DiffDegrees is min(Delta, 360.0 - Delta).

```

- **get_icing_prob**

- Si calcola il rischio di ghiaccio sulle pale.

```

get_icing_prob(X, RiskScore) :-
    get_temp(X, T), get_rh(X, H),
    T < 3.0,
    H > 80.0,
    % Più bassa è T, più alto è il rischio: (3 - T)
    % Più alta è H, più alto è il rischio: (H - 80)
    T_factor is (3.0 - T),
    H_factor is (H - 80.0),
    RiskScore is T_factor * H_factor, !.
get_icing_prob(_, 0.0).

```

○

L'estrazione e l'integrazione avviene attraverso la regola principale `kb_extract_features(X, WindScore, TurbIdx, DensScore, IcingRisk, ShearDeg)` aggrega tutte le regole sopra descritte. Lo script Python interroga questa regola per ogni istanza del dataset e mappa i risultati in cinque nuove colonne:

1. KB_WindPotential
2. KB_Turbulence
3. KB_DensityScore
4. KB_IcingProb
5. KB_Shear

```

kb_extract_features(X, WindScore, TurbIdx, DensScore, IcingRisk, ShearDeg) :-
    get_wind_score(X, WindScore),
    get_turbulence_idx(X, TurbIdx),
    get_density_score(X, DensScore),
    get_icing_prob(X, IcingRisk),
    get_shear_index(X, ShearDeg).

```

Lo script utilizzato per interrogare la KB è il seguente:

```
def query_kb_features(df_target, rules_file, facts_file): 2 usages 3 Palmdom
    prolog = Prolog()

    try:
        prolog.consult(rules_file)
        prolog.consult(facts_file)
    except:
        return df_target

    # Query unica
    query_str = "kb_extract_features(X, Wind, Turb, Dens, Ice, Shear)"
    results = list(prolog.query(query_str))

    maps = {
        'KB_WindPotential': {},
        'KB_Turbulence': {},
        'KB_DensityScore': {},
        'KB_IcingProb': {},
        'KB_Shear': {}
    }

    for res in results:
        row_key = str(res['X'])
        if row_key.startswith('r'):
            try:
                idx = int(row_key[1:])
                maps['KB_WindPotential'][idx] = float(res['Wind'])
                maps['KB_Turbulence'][idx] = float(res['Turb'])
                maps['KB_DensityScore'][idx] = float(res['Dens'])
                maps['KB_IcingProb'][idx] = float(res['Ice'])
                maps['KB_Shear'][idx] = float(res['Shear'])
            except:
                continue

    df_target['KB_WindPotential'] = df_target.index.map(maps['KB_WindPotential']).fillna(0.0)
    df_target['KB_Turbulence'] = df_target.index.map(maps['KB_Turbulence']).fillna(0.0)
    df_target['KB_DensityScore'] = df_target.index.map(maps['KB_DensityScore']).fillna(1.0)
    df_target['KB_IcingProb'] = df_target.index.map(maps['KB_IcingProb']).fillna(0.0)
    df_target['KB_Shear'] = df_target.index.map(maps['KB_Shear']).fillna(0.0)

    return df_target
```

Sono state implementate altre regole utilizzate per l'estrazione grezza dei dati:

```
get_temp(X, T)      :- meteo_data(X, T, _, _, _, _, _, _, _, _).
get_rh(X, H)        :- meteo_data(X, _, H, _, _, _, _, _, _, _).
get_wind100(X, V)    :- meteo_data(X, _, _, _, _, V, _, _, _, _).
get_gusts(X, G)      :- meteo_data(X, _, _, _, _, _, G, _, _, _).
get_wind_dir100(X, D) :- meteo_data(X, _, _, _, _, _, _, D, _, _).
get_wind_dir10(X, D) :- meteo_data(X, _, _, _, _, _, _, D, _, _).
```

L'integrazione del ragionamento logico tramite Prolog ha permesso di arricchire significativamente il dataset originale, trasformando i dati grezzi in conoscenza di dominio esplicita.

Attraverso la costruzione di una Knowledge Base composta da fatti e regole deterministiche, il sistema è riuscito a derivare nuove feature. Queste nuove colonne forniranno maggiori elementi sulla quale i modelli nella fase successiva potranno apprendere nuove relazioni per ottimizzare la previsione di energia prodotta.

5. Apprendimento supervisionato

Una fase molto fondamentale del progetto è la fase di previsione dell'energia prodotta da un sistema in un dato momento. Se durante le fasi precedenti (apprendimento non supervisionato e ragionamento logico) lo scopo era quello di cercare condizioni meteo simili fra loro e aggiungere nuove informazioni al dataset attraverso il ragionamento logico, adesso l'obiettivo è quello di andare a prevedere con esattezza l'energia prodotta.

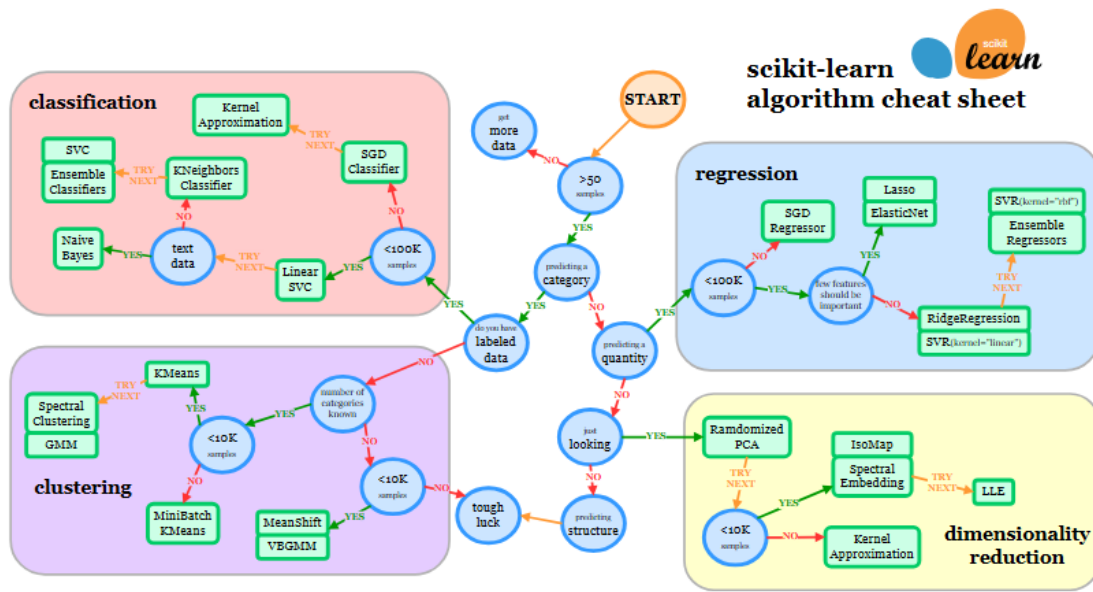
Durante questa fase è stata usata la libreria Scikit-learn, che è un framework open-source di Machine Learning per Python, basato su librerie come NumPy, SciPy e Matplotlib.

Scikit-learn

Scikit-learn offre una vasta gamma di strumenti per l'apprendimento supervisionato e non supervisionato, tra cui diversi algoritmi di:

- **Classificazione:** Questi algoritmi assegnano etichette a un'osservazione in base ai dati di input. Il risultato è discreto.
 - Esempi sono Support Vector Machine (SVM), Random Forest e K-Nearest Neighbors (KNN).
- **Clustering:** Questi algoritmi raggruppano dati simili senza etichette predefinite, ideali per scoprire pattern nascosti nei dati.
 - Esempi sono K-Means, DBSCAN e Agglomerative Clustering.
- **Riduzione della dimensionalità:** Aiuta a ridurre il numero di variabili (feature) mantenendo le informazioni più rilevanti, migliorando l'efficienza e l'interpretabilità dei modelli.
 - Esempi sono PCA (Principal Component Analysis), t-SNE e LDA (Linear Discriminant Analysis)
- **Regressione:** Utilizzati per prevedere un valore numerico continuo in base alle variabili indipendenti.
 - Esempi sono Regressione Lineare, il Random Forest Regressor e il Support Vector Regression (SVR).

Oltre a questi algoritmi, Scikit-learn è fondamentale anche per la selezione dei modelli, la valutazione delle loro prestazioni e per il preprocessing dei dati, garantendo una gestione ottimale dell'intero flusso di lavoro di Machine Learning.



RIFERIMENTO → https://scikit-learn.org/stable/machine_learning_map.html

Quale tipologia di modelli si adatta meglio ai nostri scopi?

La tipologia di modelli che meglio si adatta al nostro scopo in questo caso, sono i modelli di regressione, dal momento che essi prevedono un valore continuo, e nel nostro caso dobbiamo appunto andare a prevedere un valore numerico che è la potenza prodotta dal sistema eolico.

Algoritmi da testare

In questa fase saranno confrontati differenti algoritmi, per un confronto più ottimale fra i diversi algoritmi, gli stessi saranno testati su diversi dataset con stessa struttura ma con dati differenti. Per fare ciò sono stati presi in considerazione diversi modelli:

- **Algoritmi di Regressione Lineare**
 - Regressione Lineare Semplice → Linear regression
 - Regressione Ridge
 - Regressione Lasso
 - Regressione Elastic Net
- **Algoritmi basati su Alberi di Decisione**
 - Decision Tree Regressor
 - Random Forest Regressor
- **Algoritmi basati su Support Vector Machine**
 - Support Vector Regressor
- **Reti neurali**
 - Long Short-Term Memory

L'obiettivo è creare uno script per confrontare tutti questi modelli, e valutare quello che è il modello migliore per il nostro caso di studio.

5.1. Workflow

Il workflow per la ricerca del miglior modello predittivo prevede le seguenti fasi:

- Preparazione dei dati.
- Selezione e ottimizzazione del modello.
- Train del modello
- Valutazione delle prestazioni attraverso K-Fold.

Preparazione dei dati

Inizialmente, i dati verranno **normalizzati** per garantire che tutte le variabili siano sulla stessa scala, migliorando così l'efficacia degli algoritmi di Machine Learning. Successivamente, verrà utilizzato un **lookback** per considerare le variabili storiche al fine anche di prevedere la potenza futura, catturando così le tendenze temporali.

Selezione e l'ottimizzazione del modello

Per la selezione del miglior modello, verranno **testati diversi algoritmi**, come la regressione lineare, random forest e support vector regression (SVR), tra gli altri. Ogni modello (dove sarà possibile) sarà **ottimizzato attraverso la ricerca degli iperparametri** (come la profondità dell'albero, parametro alpha, il numero di estimatori, o i parametri di regolarizzazione), utilizzando il **Grid Search** per trovare la combinazione ottimale che massimizza le prestazioni del modello.

Valutazione delle prestazioni.

Per la valutazione del modello, verrà eseguita una valutazione delle prestazioni dei modelli, confrontando le metriche di errore, tra le quali il Mean Absolute Error (MAE) e il Root Mean Squared Error (RMSE). L'obiettivo è selezionare il modello che fornisce la previsione più precisa e affidabile per la potenza prodotta.

Le metriche utilizzate per la valutazione sono:

- **MSE (Mean Squared Error):** È la media degli errori al quadrato tra i valori previsti e quelli reali. Penalizza maggiormente gli errori più grandi ed è utile per evidenziare grosse discrepanze nei modelli.
- **RMSE (Root Mean Squared Error):** È la radice quadrata dell'MSE e fornisce un valore con la stessa unità di misura della variabile target, rendendolo più interpretabile.
- **MAE (Mean Absolute Error):** È la media del valore assoluto degli errori. A differenza dell'MSE, non penalizza in modo eccessivo gli errori più grandi, risultando più robusto in presenza di outlier.
- **Deviazione standard degli errori assoluti:** Misura quanto gli errori assoluti si discostano in media dal loro valore medio (MAE). Fornisce un'indicazione della variabilità degli errori nel modello: più è alta, più gli errori sono distribuiti in modo

irregolare; più è bassa, più gli errori sono omogenei. È utile per valutare la stabilità delle prestazioni del modello, oltre alla loro accuratezza.

Uso della Cross-Validation

Nel progetto, per la scelta del miglior modello da utilizzare, viene utilizzato il metodo **K-Fold**. Esso viene implementato impostando il parametro $k=5$. Questa tecnica serve a verificare la robustezza del modello, assicurandosi che le metriche di performance non dipendano da una specifica suddivisione dei dati. Il dataset di training viene diviso in **5 parti**: il modello viene addestrato 5 volte, utilizzando ogni volta una combinazione diversa di 4 fold per l'apprendimento e il rimanente per la validazione.

Il codice che implementa il K-Fold è il seguente:

```
for fold, (train_idx, val_idx) in enumerate(kfold.split(X)): # Kfold
    X_train_raw, X_val_raw = X[train_idx], X[val_idx]
    y_train_raw, y_val_raw = y[train_idx], y[val_idx]
    scaler_x = MinMaxScaler(feature_range=(-1, 1))
    X_train = scaler_x.fit_transform(X_train_raw)
    X_val = scaler_x.transform(X_val_raw)
    scaler_y = MinMaxScaler(feature_range=(-1, 1))
    y_train = scaler_y.fit_transform(y_train_raw)
```

A seguito delle k iterazioni effettuate della **K-Fold Cross Validation** otteniamo un report. Il report contiene la media delle metriche precedentemente descritte, ovvero:

- **RMSE**
- **MAE**
- **MSE**
- **Std. Err**

6. Preparazione dei dati

Nella prima fase dello script i dati vengono organizzati in modo da essere adatti al train.

5.1. Organizzazione dei dati

Funzione di lookback

I modelli che andremo a testare accettano dati con la seguente struttura ($n_samples$, $n_features$), quindi dati bidimensionali a differenza delle reti neurali (es. LSTM) che prendono in input dati a 3 dimensioni ($n_samples$, $lookback$, $n_features$).

Dal momento che andremo ad applicare una funzione di lookback dobbiamo cercare un modo per trasformarli da 3 dimensioni a 2.

Questo viene fatto con la seguente funzione:

```
def create_lookback_features(df, target_column, lookback):
    """Aggiunge colonne di lookback al dataset."""
    for i in range(1, lookback + 1):
        df[f"{i}"] = df[target_column].shift(i)
    df.dropna(inplace=True)
    return df
```

La funzione aggiunge colonne che rappresentano i valori precedenti ($lookback$) della colonna di destinazione ($target_column$), che nel nostro caso è la potenza. Per ogni passo temporale da 1 fino a $lookback$, essa crea una nuova colonna contenente i valori della colonna di destinazione (potenza) spostati indietro nel tempo di i passi.

Cos'è il lookback?

Essa definisce il numero di passi temporali passati o osservazioni precedenti (nel nostro caso le rilevazioni precedenti) che il modello dovrebbe prendere in considerazione per fare previsioni. Quindi il $lookback$ determina quante informazioni storiche devono essere incluse nel modello per catturare le dipendenze temporali.

Esempio → Nel nostro caso sono state usate anche le 50 precedenti rilevazioni per effettuare la predizione.

Divisione dei dati in train e test

Nel nostro caso dato che viene usato il K-Fold per la fase di valutazione del modello la divisione del dataset in train e test avviene attraverso la classe `Kfold`.

```
kfold = KFold(n_splits=n_splits, shuffle=True)
```

Essa ci permette di creare n_split . Dove $n_split - 1$ saranno usati per il train e uno split per la fase di test di quella specifica iterazione. Questa classe ci garantisce che i dati di training e di testing siano completamente separati, in modo che i modelli non possano "vedere" i dati di test

durante l'allenamento. Questo assicura che le valutazioni del modello siano accurate e riflettono realmente la sua capacità di generalizzare a nuovi dati, evitando il rischio di overfitting e garantendo che le metriche di performance siano valide.

5.2. Normalizzazione dei dati

```
X_train_raw, X_val_raw = X[train_idx], X[val_idx]
y_train_raw, y_val_raw = y[train_idx], y[val_idx]
scaler_x = MinMaxScaler(feature_range=(-1, 1))
X_train = scaler_x.fit_transform(X_train_raw)
X_val = scaler_x.transform(X_val_raw)
scaler_y = MinMaxScaler(feature_range=(-1, 1))
y_train = scaler_y.fit_transform(y_train_raw)
```

La normalizzazione dei valori è un processo che consente di ridurre o eliminare le discrepanze tra le scale numeriche di diverse caratteristiche in un dataset. Viene applicata per garantire che tutte le caratteristiche abbiano lo stesso ordine di grandezza e, per trasformare i dati in un intervallo specifico. Nel nostro caso è stato utilizzato il range da -1 a 1.

7. Ottimizzazione iperparametri

7.1. Algoritmi di Regressione Lineare

Per questa tipologia di algoritmi abbiamo confrontato 4 tipologie:

- Regressione Lineare Semplice
 - LinearRegression()
- Regressione Ridge
 - Ridge()
- Regressione Lasso
 - Lasso()
- Regressione Elastic Net
 - ElasticNet()

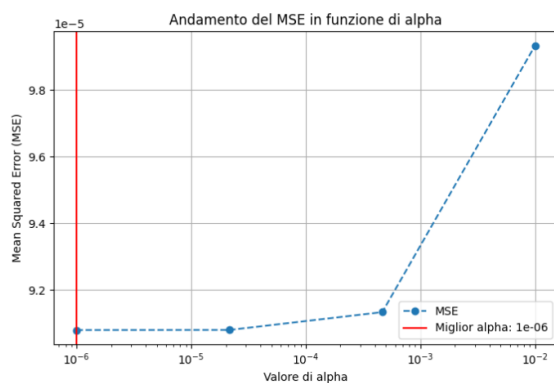
7.1.1. Parametro alpha

Alpha essa controlla la penalizzazione applicata ai coefficienti della regressione lineare.

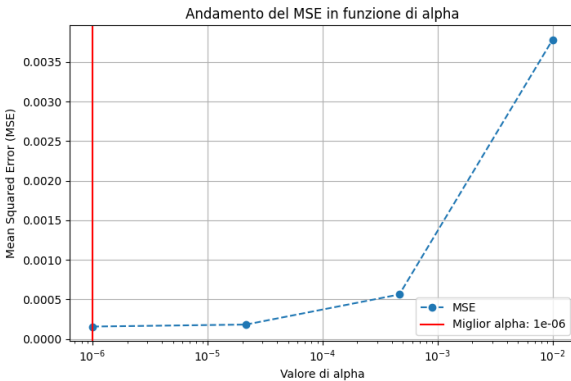
- **alpha alto** Maggiore penalizzazione → i coefficienti vengono ridotti di più, riducendo l'overfitting.
- **alpha basso** Minore penalizzazione → il modello si avvicina alla regressione lineare standard, adattandosi maggiormente ai dati.

Per ottimizzare il valore di alpha, abbiamo utilizzato la classe GridSearchCV, che permette di trovare il miglior parametro per l'algoritmo utilizzato. Gli algoritmi testati per migliorare il parametro Alpha sono stati:

- Ridge.
 - **CONCLUSIONI** → Secondo questo grafico il metodo parametro migliore di alfa e 10^{-6}



- Lasso
 - **CONCLUSIONE** → Secondo questo grafico il metodo parametro migliore di alfa è 10^{-6}



CONCLUSIONE → Il valore di alpha che offre le migliori performance per entrambi i modelli (Ridge e Lasso) è 10^{-6} . Man mano che alpha si avvicina a zero, i modelli tendono a comportarsi in modo simile alla regressione lineare standard.

7.1.2. Parametro l1_ratio

Il parametro l1_ratio in ElasticNet controlla il bilanciamento tra le **penalizzazioni L1 (Lasso) e L2 (Ridge)**:

- **l1_ratio = 0** → ElasticNet equivale a una Ridge Regression pura, applicando solo la penalizzazione L2.
- **l1_ratio = 1** → ElasticNet si comporta come una Lasso Regression, applicando solo la penalizzazione L1.
- **$0 < \text{l1_ratio} < 1$** → ElasticNet combina entrambe le penalizzazioni, dove valori più vicini a 1 danno maggiore peso alla L1 (selezione delle feature), mentre valori più vicini a 0 favoriscono la L2 (riduzione della varianza).

L'uso di l1_ratio permette di **ottenere un compromesso tra la riduzione della dimensionalità (Lasso) e la stabilizzazione dei coefficienti (Ridge)**, rendendo ElasticNet adatto a problemi con dati correlati o con molte feature irrilevanti.

ElasticNet

Questo algoritmo ha subito l'ottimizzazione di 2 parametri attraverso la Grid Search:

- Alpha
- l1_ratio

I risultati dei test mostrano che il **miglior valore di alpha è 10^{-6}** , poiché minimizza l'errore per quasi tutti i valori di `l1_ratio`. Inoltre, i risultati indicano che un **`l1_ratio` compreso tra 0.1 e 0.5 garantisce prestazioni ottimali**, bilanciando la penalizzazione L1 (selezione delle feature) e L2 (stabilizzazione dei coefficienti), rendendo il modello più robusto e generalizzabile^{5.2}. Algoritmi basati su Alberi di Decisione

Per questa tipologia di algoritmi abbiamo confrontato 2 tipologie:

- Decision Tree Regressor
 - `DecisionTreeRegressor()`
- Random Forest Regressor
 - `RandomForestRegressor()`

7.2.1. Decision Tree Regressor

È un modello che costruisce un albero decisionale per fare previsioni basate su tutte le variabili del dataset.

7.2.2. Random Forest Regressor

È un insieme di molti alberi decisionali. Ogni albero prende decisioni su sottoinsiemi di dati e caratteristiche, migliorando la previsione della potenza dell'impianto eolico. La previsione finale è la media dei risultati ottenuti da tutti gli alberi.

7.2. Algoritmi basati su Support Vector Machine

Per questa tipologia di algoritmi abbiamo analizzato il Support Vector Regressor fornitoci dalla classe `SVR()`.

L'obiettivo di SVR è trovare una funzione che mappi le variabili di input a valori continui, minimizzando l'errore tra i valori previsti e quelli reali.

Parametri

- **C (penalty parameter):** Controlla la penalizzazione degli errori. Un valore alto di C tende a dare un modello più complesso, mentre un valore basso consente maggiore flessibilità.
- **epsilon (ϵ):** Determina la larghezza del margine di tolleranza in cui l'errore non viene penalizzato. Un valore troppo grande può ridurre l'accuratezza, mentre uno troppo piccolo può portare a overfitting.

- **kernel:** Tipo di kernel da utilizzare per trasformare i dati. I principali sono linear, poly, rbf (Radial Basis Function), e sigmoid.

7.3. LSTM model

Le Long Short-Term Memory (LSTM) sono una classe di reti neurali ricorrenti (RNN) progettate per gestire serie temporali e dati sequenziali. Grazie alla loro architettura con gates (ingressi di controllo), le LSTM riescono a memorizzare informazioni a lungo termine e a mitigare il problema della scomparsa del gradiente, tipico delle RNN tradizionali.

Struttura del modello

Il modello testato è un modello molto semplice, esso prevede la seguente struttura.

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 1, 128)	92672
lstm_1 (LSTM)	(None, 64)	49408
dense (Dense)	(None, 25)	1625
dense_1 (Dense)	(None, 1)	26

```

=====
Total params: 143,731
Trainable params: 143,731
Non-trainable params: 0

```

Risultati

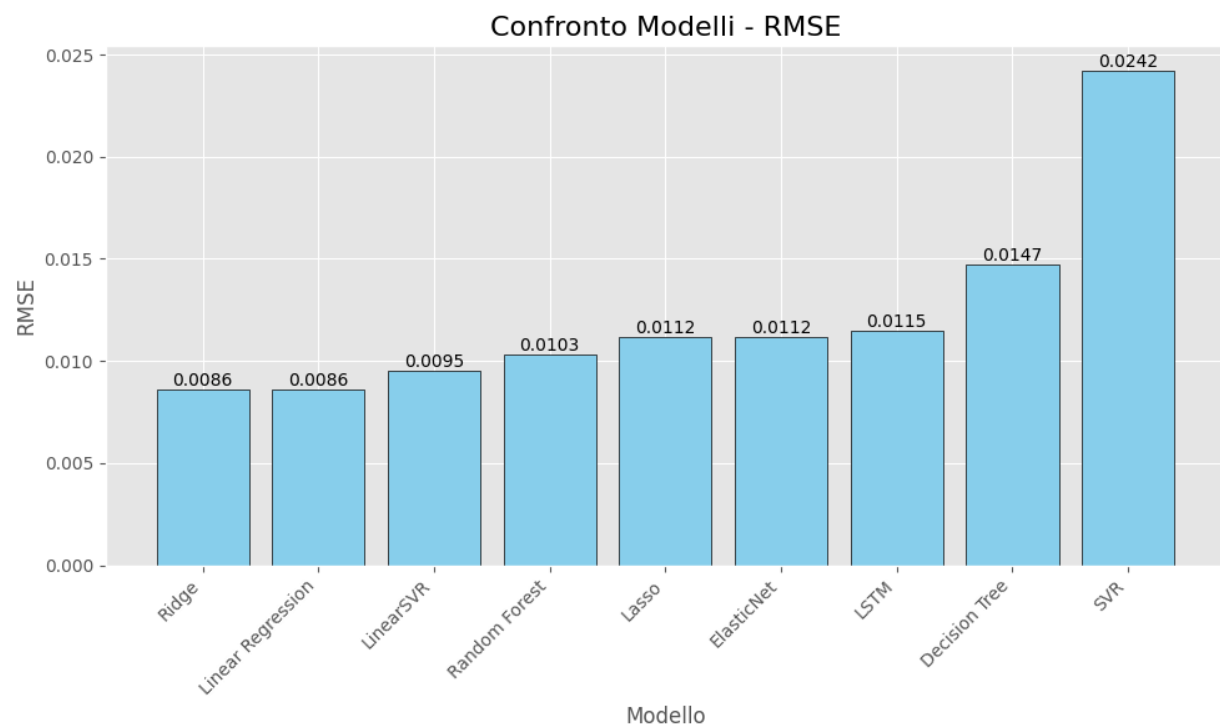
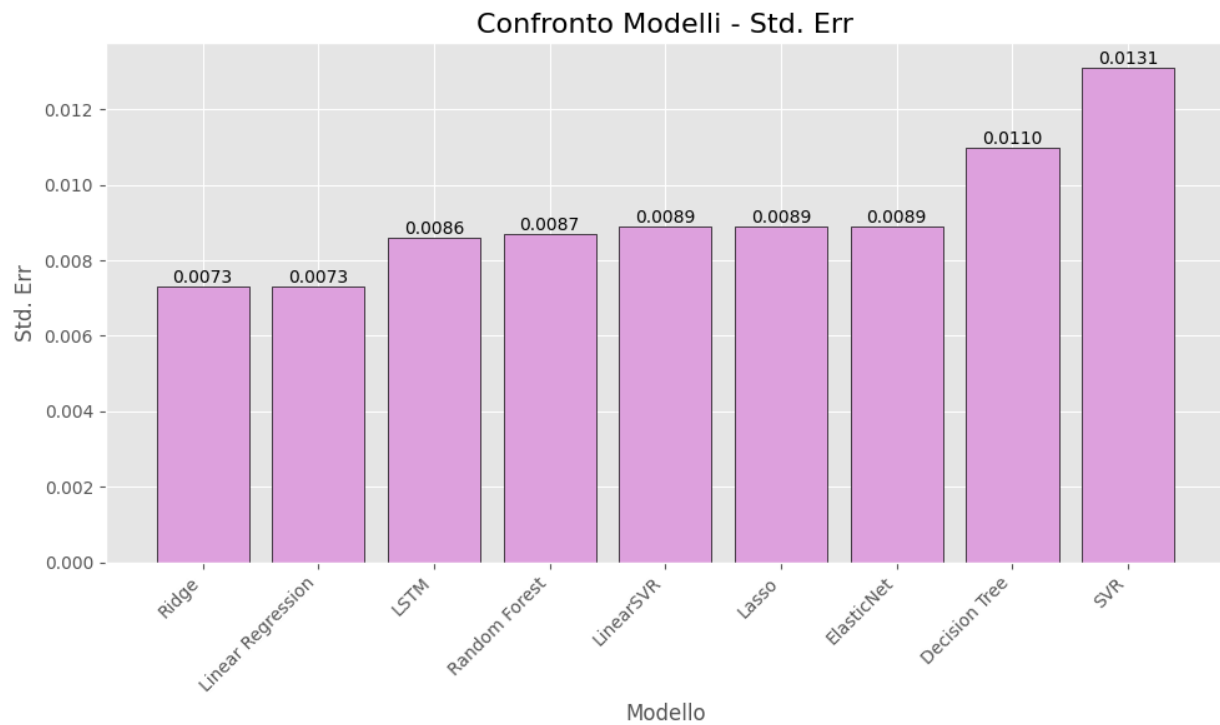
I risultati ottenuti sono molto scarsi in confronto a quelli degli algoritmi di regressione. Questi modelli hanno un grande potenziale, le reti neurali e gli LSTM possono offrire ottimi risultati, ma richiedono molta potenza di calcolo e tempo. Questi modelli necessitano di un'accurata ricerca per trovare la migliore architettura, come il numero di livelli e il tipo di funzioni di attivazione da usare. **La fase di ottimizzazione può richiedere tempo, soprattutto nel caso si utilizzino grandi quantità di dati.** In conclusione, mentre gli algoritmi di regressione sono più semplici e veloci, le **reti neurali** anche se più complesse, una volta ottimizzate, potrebbero **fornire previsioni più accurate** sulla potenza generata dagli impianti eolici.

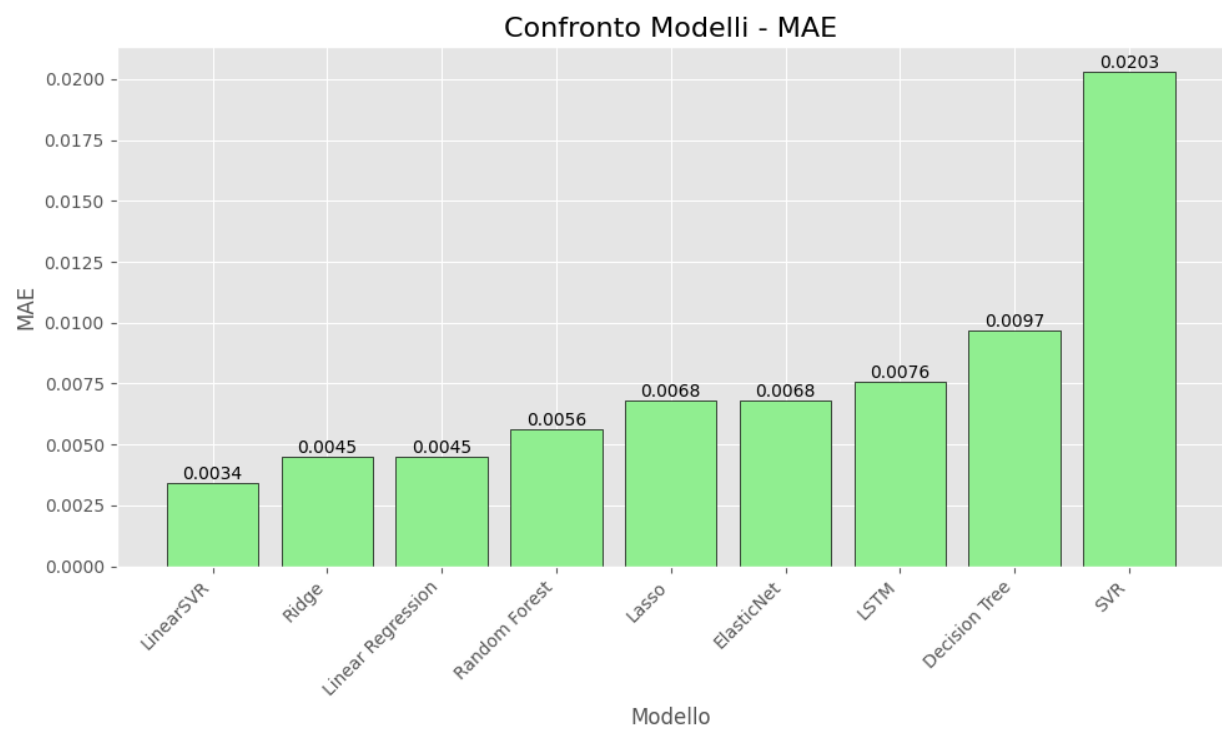
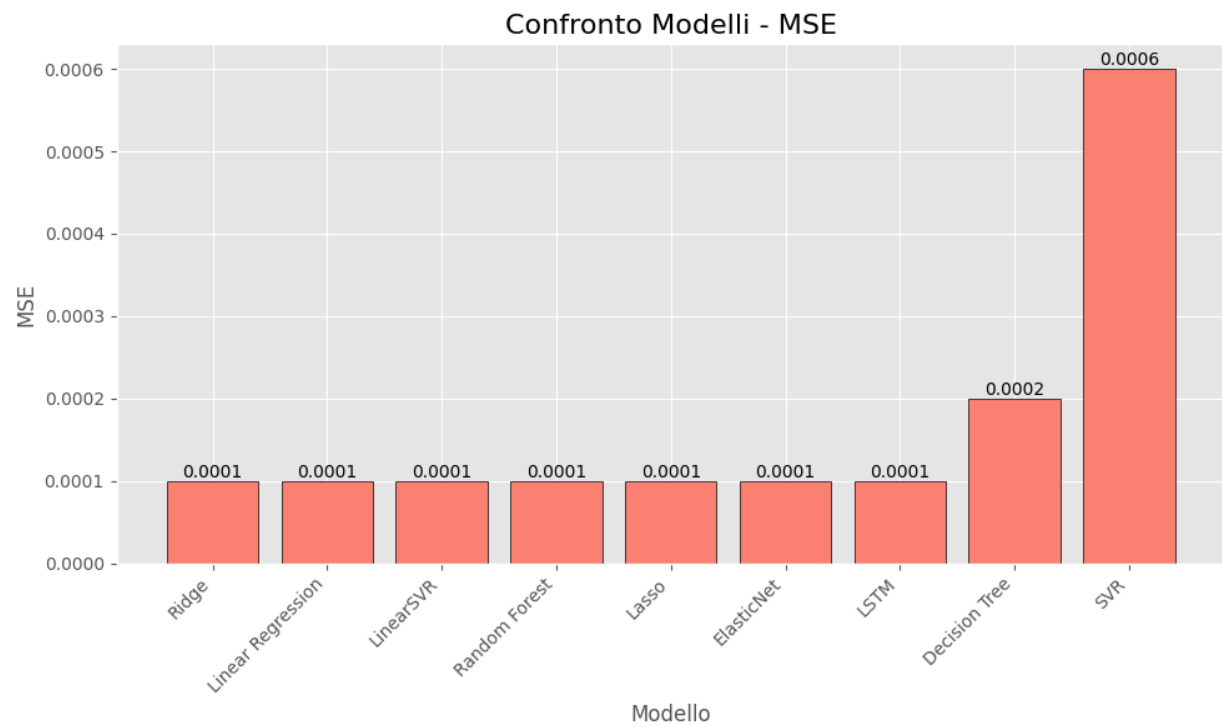
8. Confronto modelli

Tabella di confronto delle prestazioni dei modelli:

Modello	RMSE	MAE	MSE	Std. Err
Ridge	0.0086 (± 0.000)	0.0045	0.0001	0.0073
Linear Regression	0.0086 (± 0.000)	0.0045	0.0001	0.0073
LinearSVR	0.0095 (± 0.000)	0.0034	0.0001	0.0089
Random Forest	0.0103 (± 0.000)	0.0056	0.0001	0.0087
Lasso	0.0112 (± 0.000)	0.0068	0.0001	0.0089
ElasticNet	0.0112 (± 0.000)	0.0068	0.0001	0.0089
LSTM	0.0115 (± 0.001)	0.0076	0.0001	0.0086
Decision Tree	0.0147 (± 0.000)	0.0097	0.0002	0.0110
SVR	0.0242 (± 0.000)	0.0203	0.0006	0.0131

Grafici che mostrano le diverse metriche ottenute dai modelli:





8.2. Analisi dei risultati

Analizzando i risultati riportati nella tabella e visualizzati nei grafici a barre, è possibile delineare una chiara gerarchia di prestazioni tra gli algoritmi testati. L'analisi si basa sul confronto incrociato delle metriche di errore e stabilità.

I modelli di regressione lineare sembrano essere i più performanti. In particolare, Ridge e Linear Regression ottengono il miglior punteggio sulla metrica RMSE, i quali riescono quindi a minimizzare meglio gli errori quadratici medi. Questo viene dimostrato anche nelle altre metriche come la Std. Err dove ottengono il punteggio di 0.0073, il che suggerisce che questi modelli mantengono prestazioni costanti attraverso i diversi *fold* della cross-validation.

Anche il modello LinearSVR risulta ottimo nonostante abbia un RMSE leggermente superiore ai modelli precedentemente visti dal momento che registra il MAE più basso in assoluto. Questo ci dice che il LinearSVR è estremamente preciso sulla media delle predizioni, ma commette occasionalmente errori di entità maggiore che vengono penalizzati dalla metrica RMSE.

Performance dei Modelli basati su Alberi Tra i metodi basati su alberi decisionali, il Random Forest supera nettamente il singolo Decision Tree.

I modelli più complessi come LSTM e SVR non hanno prodotto i risultati migliori. Il modello LSTM ottiene un RMSE di 0.0115. Nonostante le scarse prestazioni del modello LSTM, esso potrebbe diventare un ottimo candidato ad essere il miglior modello nel momento in cui si estende il dataset e/o si crei un modello LSTM più robusto e profondo. Il modello SVR invece è quello meno performante, con il peggior RMSE (0.0242) e il peggior MAE (0.0203).

Riferimenti Bibliografici

- www.kaggle.com
- <https://scikit-learn.org/stable/>
- https://scikit-learn.org/stable/machine_learning_map.html